



Proceso de Desarrollo de Software

TPO - Grupo 07

Profesor: Ruiz Matias Gabriel, Cuomo Ghio Pedro

Comisión: Jueves Noche

Integrantes:

Nº	Apellido y Nombre	Carrera	Mail	Legajo
1	De la Torre, Sofia	Ingeniería Informática	sodelatorre@uade.edu.ar	1200753
2	Fiumana, Valentina	Ingeniería Informática	vfiumana@uade.edu.ar	1201817
3	Frechero, Nahuel	Ingeniería Informática	nfrechero@uade.edu.ar	1170867
4	Pedreira, Tomas	Ingeniería Informática	tpedreira@uade.edu.ar	1160273
5	Mendoza, Lucas	Ingeniería Informática	lmendozabeccar@uade.edu.ar	1191022
6	Ramos, Lisandro	Ingeniería Informática	lisaramos@uade.edu.ar	1191701

eScrims - Fundamentación exhaustiva de patrones de diseño.....	4
Marco de análisis SOLID y GRASP aplicado al diseño.....	4
Principios SOLID.....	5
Principios GRASP.....	6
Patrón State.....	7
Problema si no se aplica el patrón.....	7
Solución aplicada en el diseño.....	7
Flujo de ejemplo dentro de eScrims.....	8
Beneficios técnicos.....	8
Relación con SOLID y GRASP.....	9
Patrón Strategy.....	10
Problema si no se aplica el patrón.....	10
Solución aplicada en el diseño.....	10
Flujo de ejemplo dentro de eScrims.....	11
Beneficios técnicos.....	11
Relación con SOLID y GRASP.....	11
Patrón Observer.....	12
Problema concreto si no se aplica el patrón.....	12
Solución aplicada en el diseño.....	12
Flujo de ejemplo dentro de eScrims.....	13
Beneficios técnicos.....	13
Relación con SOLID y GRASP.....	14
Patrón Abstract Factory.....	15
Problema si no se aplica el patrón.....	15
Solución aplicada en el diseño.....	15
Flujo de ejemplo dentro de eScrims.....	16

Beneficios técnicos.....	16
Relación con SOLID y GRASP.....	16
Patrón Builder.....	18
Problema concreto si no se aplica el patrón.....	18
Solución aplicada en el diseño.....	18
Flujo de ejemplo dentro de eScrims.....	18
Beneficios técnicos.....	19
Relación con SOLID y GRASP.....	19
Patrón Command.....	20
Problema si no se aplica el patrón.....	20
Solución aplicada en el diseño.....	20
Flujo de ejemplo dentro de eScrims.....	21
Beneficios técnicos.....	21
Relación con SOLID y GRASP.....	21
Patrón Adapter.....	22
Problema si no se aplica el patrón.....	22
Solución aplicada en el diseño.....	22
Flujo de ejemplo dentro de eScrims.....	23
Beneficios técnicos.....	23
Relación con SOLID y GRASP.....	23
Patrón Chain of Responsibility.....	25
Problema si no se aplica el patrón.....	25
Solución aplicada en el diseño.....	25
Flujo de ejemplo dentro de eScrims.....	26
Casos de Uso.....	27
Sistema eScrims - Gestión de Usuario y Búsqueda de Scrims.....	27

Sistema eScrims — Gestión del scrim y notificaciones.....	28
Sistema eScrims — Casos de uso de moderación y reportes.....	29
Diagrama general del caso de uso.....	30
Diagrama de Estados - Ciclo de vida de un Scrim.....	31
Historias de Usuario - Plataforma eScrim.....	32

Fundamentación del diseño orientado a objetos: Plataforma eScrims

Introducción al diseño

El sistema eScrims fue diseñado aplicando principios de diseño orientado a objetos buscando obtener una arquitectura modular, extensible y con bajo acoplamiento. El dominio representa una plataforma para organizar scrims de distintos eSports, permitiendo gestionar usuarios, perfiles de juego, creación de encuentros, matchmaking, equipos, estados del scrim, notificaciones, moderación y servicios externos.

A partir del análisis del dominio se identificaron diferentes responsabilidades que fueron separadas en módulos:

- **Módulo Usuario:** administra la identidad del usuario, perfiles de juego, preferencias, búsquedas favoritas y datos asociados.
- **Módulo Jugador:** representa la participación del usuario dentro de un contexto competitivo, incluyendo equipos, roles, rangos y estadísticas.
- **Módulo Juego:** encapsula las reglas particulares de cada eSport y sus configuraciones.
- **Módulo Scrim:** concentra el ciclo de vida del encuentro, postulaciones, confirmaciones, equipos y resultados.
- **Módulo Matchmaking:** contiene la lógica de selección de jugadores mediante estrategias intercambiables.
- **Módulo Calendario:** permite integración con servicios externos para sincronización de eventos.
- **Módulo Notificaciones:** desacopla los eventos del dominio de los distintos canales de comunicación.
- **Módulo Moderación:** administra reportes de conducta, penalidades y escalamiento de casos.

La separación en módulos permite que cada componente tenga responsabilidades claras y evita que entidades principales como Usuario o Scrim concentren lógica que pertenece a otros servicios.

Aplicación de SOLID y GRASP

El diseño utiliza principios SOLID y GRASP para controlar la complejidad del sistema.

SRP: Single Responsibility Principle

Las clases poseen una única responsabilidad:

- Usuario administra información del usuario.
- PerfilJuego mantiene la relación del usuario con un juego específico.
- BusquedaFavorita representa criterios guardados de búsqueda.
- Scrim representa la información fija del evento
- Lobby gestiona las postulaciones y confirmaciones
- ScrimFacade gestiona las operaciones relacionadas al Scrim
- LobbyFacade gestiona las operaciones relacionadas al Lobby
- ModeracionFacade gestiona las operaciones relacionadas a Moderación
- GestorLobby gestiona lo relacionado a los equipos y roles
- ScrimCommandHistory administra historial de acciones
- ScrimLifecycleService administra las transiciones del ciclo de vida.
- MatchmakingService coordina la selección de jugadores.
- IMatchmakingStrategy representa únicamente la lógica de selección.
- DomainEventBus administra publicación y distribución de eventos.
- NotificationObserver reacciona a eventos generando comunicaciones.
- NotificationService gestiona el envío de notificaciones.
- INotifierFactory centraliza la creación de notificadores.
- ModerationService coordina resolución de reportes.
- Cada ModerationHandler resuelve únicamente un nivel de moderación.

Esto evita que clases como Scrim, Usuario o Service acumulen responsabilidades no relacionadas.

OCP: Open/Closed Principle

El sistema permite agregar funcionalidades nuevas mediante extensión:

- Nuevos estados del scrim agregando nuevas implementaciones de ScrimState.
- Nuevos algoritmos de matchmaking agregando nuevas implementaciones de IMatchmakingStrategy.
- Nuevos canales de comunicación agregando nuevos Notifier.
- Nuevos proveedores externos agregando nuevos Adapter.
- Nuevas acciones del lobby agregando nuevos ScrimCommand.
- Nuevos niveles de moderación agregando nuevos ModerationHandler.

Las clases existentes no necesitan modificarse ante estos cambios.

LSP: Liskov Substitution Principle

Las implementaciones concretas pueden reemplazar sus abstracciones.

Ejemplos:

- BuscandoJugadoresState, LobbyArmadoState, ConfirmadoState, EnJuegoState, FinalizadoState y CanceladoState pueden utilizarse como ScrimState.
- ByMMRStrategy, ByLatencyStrategy y ByHistoryStrategy pueden utilizarse como IMatchmakingStrategy.
- DevNotifierFactory y ProdNotifierFactory cumplen el contrato de INotifierFactory.
- AutoModerationHandler, BotModerationHandler y HumanModerationHandler cumplen el comportamiento esperado de ModerationHandler.
- Jugador y Moderador cumplen con las características de Usuario

ISP: Interface Segregation Principle

Las interfaces están separadas según responsabilidad:

- ScrimState contiene operaciones propias del ciclo de vida.
- IMatchmakingStrategy contiene selección de jugadores.
- LobbyCommand contiene ejecutar y deshacer.
- INotifier contiene envío de notificaciones.
- ICalendarAdapter contiene operaciones de calendario.
- IAdapterAuth contiene la operacion de autenticar

Esto evita interfaces grandes que obliguen a implementar métodos innecesarios.

DIP: Dependency Inversion Principle

Las clases de alto nivel dependen de abstracciones:

- MatchmakingService depende de IMatchmakingStrategy.
- NotificationObserver depende de INotifierFactory.
- OAuthManager depende de IAdapterAuth.
- CalendarService depende de ICalendarAdapter.

Las integraciones externas como:

- Steam
- Riot
- Discord
- SendGrid
- JavaMail
- calendarios externos

son encapsuladas mediante adapters.

Principios GRASP aplicados

Information Expert

La responsabilidad queda ubicada donde existe la información necesaria:

- ScrimState conoce las reglas de cada estado.
- ScrimBuilder conoce los datos necesarios para construir un scrim válido.
- Juego conoce sus reglas particulares.
- Penalidad conoce la información de sanción aplicada.
- Postulacion sabe como aceptarse o rechazarse
- Equipo sabe cómo operar con sus propios jugadores

Creator

Las clases crean objetos cuando poseen la información necesaria:

- ScrimBuilder crea Scrim.
- INotifierFactory crea notificadores.
- Los eventos son creados cuando ocurre una modificación relevante del dominio.
- UsuarioFactory crea Jugador o Moderador
- Lobby crea las postulaciones y confirmaciones cuando es delegado por ScrimFacade

Controller

Los servicios coordinan casos de uso:

- ScrimLifecycleService
- MatchmakingService

- NotificationService
- ModerationService
- CalendarService

reciben solicitudes y delegan comportamiento.

Low Coupling

Se evita dependencia directa:

- Scrim no conoce servicios externos.
- NotificationObserver no conoce proveedores concretos.
- MatchmakingService no conoce algoritmos específicos.
- ModerationService no conoce cada regla individual.
- Jugador no conoce mas que a los Facades

High Cohesion

Cada componente concentra una responsabilidad:

- Estados manejan estados.
- Strategies manejan algoritmos.
- Commands manejan acciones.
- Adapters manejan traducciones externas.
- Handlers manejan niveles de moderación.
- Facades manejan subsistemas relacionados

Polymorphism

El sistema reemplaza condicionales mediante objetos:

- Estados del scrim.
- Estrategias de matchmaking.
- Comandos.
- Notificadores.
- Handlers de moderación.

Pure Fabrication

Se incorporan clases técnicas que no representan entidades del negocio pero mejoran el diseño:

- DomainEventBus
- ScrimBuilder
- NotificationService
- CalendarService
- INotifierFactory
- UsuarioFactory
- CommandHistoryInvoker
- Facades

Protected Variations

Se protegen puntos de cambio frecuentes:

- Ciclo de vida del scrim.
- Algoritmos de matchmaking.
- Proveedores externos.
- Canales de comunicación.
- Reglas de moderación.
- Autenticación
- Comandos del lobby

Patrones aplicados según el diagrama

Patrón State

Aspecto	Detalle
Clases UML	ScrimState, BuscandoJugadoresState, LobbyArmadoState, ConfirmadoState, EnJuegoState, FinalizadoState, CanceladoState, ScrimLifecycleService y Scrim como entidad/contexto de dominio.
Requerimientos relacionados	RF4 Estados del scrim; diagrama de estados; transiciones automáticas por tiempo mediante scheduler; trazabilidad de cambios de estado.

Problema

El scrim no es un objeto estático: cambia de comportamiento según la fase en la que se encuentre. En BuscandoJugadores acepta postulaciones; en LobbyArmado bloquea nuevas postulaciones y exige confirmaciones; en Confirmado espera la fecha/hora; en EnJuego permite finalizar; en Finalizado habilita estadísticas; y en Cancelado bloquea casi todo. Si se usara un atributo estado de tipo String o enum con muchos if/switch dentro de ScrimLifecycleService o Scrim, cada operación tendría que preguntar permanentemente en qué estado se encuentra el scrim. Si todas estas reglas estuvieran dentro de Scrim, se generaría una clase con muchos condicionales y difícil mantenimiento. El problema se agrava porque la consigna incluye transiciones automáticas por tiempo y reglas de negocio. Un cambio pequeño, por ejemplo agregar un estado Suspendido, Reprogramado o EnDisputa, obligaría a modificar varios métodos ya probados.

Solución aplicada

Se utiliza:

- ScrimState
- BuscandoJugadoresState
- LobbyArmadoState
- ConfirmadoState
- EnJuegoState
- FinalizadoState
- CanceladoState

Cada estado conoce qué operaciones puede realizar y cuáles son inválidas. ScrimLifecycleService actúa como coordinador del cambio de estado.

ScrimState define una interfaz común para avanzar(scrim) y cancelar(scrim). En una versión más completa también puede incluir postular, confirmar, iniciar, finalizar y cargarEstadistica, pero en el UML actual aparecen avanzar y cancelar como operaciones comunes. Cada estado concreto concentra las reglas permitidas para esa fase. BuscandoJugadoresState decide si el cupo se completó y, en ese caso, solicita el cambio a LobbyArmadoState. LobbyArmadoState valida confirmaciones y puede avanzar a ConfirmadoState. ConfirmadoState no debería iniciar manualmente antes de tiempo, sino esperar al scheduler o a ScrimLifecycleService cuando fechaHora sea alcanzada. EnJuegoState permite finalizar. FinalizadoState y CanceladoState actúan como estados terminales. ScrimLifecycleService conserva la

referencia estado: ScrimState y expone operaciones como cambiarEstado, cancelar, finalizar e iniciar. Esto ayuda a que Scrim no absorba toda la lógica de ciclo de vida y evita que se convierta en una god class. Cada transición debería publicar un DomainEvent mediante DomainEventBus: LobbyArmadoEvent, ConfirmadoEvent, EnJuegoEvent, CanceladoEvent o eventos equivalentes. Así State queda conectado con Observer sin acoplarse a notificaciones concretas.

Beneficios

- Evita grandes bloques if/else.
- Permite agregar nuevos estados.
- Centraliza reglas de transición.
- Facilita pruebas unitarias.

Flujo de ejemplo dentro de eScrims

1. Un usuario se postula mientras el scrim está en BuscandoJugadoresState.
2. Cuando se completa el cupo, BuscandoJugadoresState invoca cambiarEstado(new LobbyArmadoState()).
3. El cambio publica LobbyArmadoEvent para notificar a participantes.
4. Cada jugador confirma. Cuando todos confirmaron, LobbyArmadoState cambia a ConfirmadoState y se emite ConfirmadoEvent.
5. Al llegar la fecha/hora, el scheduler llama a iniciar. ConfirmadoState cambia a EnJuegoState y publica EnJuegoEvent.
6. Al terminar, EnJuegoState cambia a FinalizadoState y se habilitan estadísticas.

Relación con SOLID y GRASP

- SOLID: State respeta SRP porque cada clase de estado tiene una única razón de cambio: las reglas y transiciones de esa fase. BuscandoJugadoresState cambia si cambian las reglas para completar cupos; LobbyArmadoState cambia si cambian las confirmaciones; FinalizadoState cambia si cambian las reglas de carga de estadísticas. Sin este patrón, todas esas razones de cambio caerían juntas sobre Scrim o ScrimLifecycleService.

- SOLID: también aplica OCP porque agregar un estado nuevo, por ejemplo SuspendidoState o PausadoState, no debería obligar a editar todos los if/switch del ciclo de vida. Se agrega una nueva clase que implementa ScrimState y se configura la transición correspondiente. DIP aparece porque ScrimLifecycleService trabaja contra la abstracción ScrimState y no contra clases concretas.
- GRASP: el patrón usa Polymorphism para reemplazar condicionales por comportamiento distribuido en objetos. También mejora High Cohesion, ya que cada estado concentra únicamente la lógica de su etapa. Information Expert se cumple porque el objeto estado es el experto en responder qué operaciones son válidas en esa fase. Protected Variations protege al sistema ante cambios en el flujo del scrim, uno de los puntos más variables de la consigna.

Patrón Strategy

Aspecto	Detalle
Clases UML	IMatchmakingStrategy, ByMMRStrategy, ByLatencyStrategy, ByHistoryStrategy, MatchmakingService y Scrim mediante selección de estrategia.
Requerimientos relacionados	RF5 Estrategias de emparejamiento; objetivo de emparejar por rango/MMR, rol y latencia; RNF de rendimiento menor a 2s para 500 candidatos.

Problema

El matchmaking es una regla central del sistema y no existe una única manera correcta de elegir jugadores. Un scrim competitivo puede priorizar MMR; uno casual puede priorizar latencia; y uno de práctica puede ponderar historial, roles, fair play y abandono previo. Sin Strategy, MatchmakingService tendría un bloque if/else con un flag tipo criterio = MMR/LATENCIA/HISTORIAL. Ese diseño mezcla algoritmos distintos en un mismo método, dificulta testear rendimiento por separado y vuelve riesgoso agregar una estrategia híbrida. La consigna pide algoritmos intercambiables. Si el algoritmo queda fijo dentro del servicio, el sistema no cumple con esa flexibilidad.

Solución aplicada en el diseño

- IMatchmakingStrategy define seleccionar(candidatos: List): List. En el esqueleto de la consigna se sugiere además recibir Scrim como parámetro, lo cual es recomendable porque la estrategia necesita conocer rangoMin, rangoMax, región, latenciaMax, roles y formato.
- MatchmakingService mantiene estrategia: IMatchmakingStrategy y expone setEstrategia(s). El service solo invoca estrategia.seleccionar(candidatos, scrim) y no conoce los detalles internos del cálculo.
- ByMMRStrategy usa el MMR/rango para filtrar jugadores dentro del rango del scrim y controlar la diferencia máxima configurable.
- ByLatencyStrategy usa latenciaMaxima y región para priorizar jugadores con ping aceptable.

- ByHistoryStrategy pondera pesoMMR, pesoLatencia y pesoHistorial. En el UML aparece con pesos, por lo que conviene fundamentarla como estrategia compuesta de compatibilidad, no como un simple filtro.
- Una futura HybridStrategy podría combinar MMR + latencia + abandono previo sin modificar MatchmakingService.

Flujo de ejemplo dentro de eScrims

7. El organizador crea un scrim con región, rango y latencia máxima.
8. MatchmakingService recibe 500 candidatos.
9. Según configuración del scrim, se asigna ByMMRStrategy, ByLatencyStrategy o ByHistoryStrategy.
10. La estrategia devuelve candidatos ordenados o filtrados.
11. El lobby se arma respetando cupos y roles.

Beneficios técnicos

- Permite cambiar algoritmo por tipo de scrim sin tocar el dominio.
- Hace posible medir rendimiento por estrategia con pruebas de carga.
- Facilita AB testing o configuración por juego/región.
- Evita que MatchmakingService crezca con condicionales.
- Respeta Abierto/Cerrado y Responsabilidad Única.

Relación con SOLID y GRASP

- SOLID: Strategy aplica SRP porque cada algoritmo de matchmaking tiene una responsabilidad aislada: MMR, latencia o historial. Esto evita que MatchmakingService tenga que conocer todos los detalles matemáticos o heurísticos de cada criterio. OCP es central: si mañana se agrega HybridStrategy, AvailabilityStrategy o RegionPriorityStrategy, se incorpora una clase nueva sin modificar las estrategias existentes.

- SOLID: LSP es importante porque todas las estrategias deben poder usarse donde se espera `IMatchmakingStrategy`. `MatchmakingService` no debería necesitar preguntar si la estrategia es de MMR, latencia o historial. DIP se cumple porque el servicio depende de una interfaz, no de una implementación concreta. ISP se mantiene si la interfaz solo exige seleccionar candidatos y no mezcla responsabilidades de persistencia, notificación o logging.
- GRASP: Strategy es un caso claro de Polymorphism y Protected Variations, porque el algoritmo de selección es un punto que cambia según juego, región, modo competitivo/casual o rendimiento requerido. También mejora Low Coupling: el servicio no queda acoplado a fórmulas concretas. High Cohesion aparece porque cada estrategia concentra su propio criterio y puede probarse con datos controlados.

Patrón Observer

Aspecto	Detalle
Clases UML	IObservable, IObservable, DomainEventBus, DomainEvent, NotificationObserver, AlertaBusquedaObserver, BusquedaFavorita, NotificationService, eventos ScrimCreadoEvent, LobbyArmadoEvent, ConfirmadoEvent, EnJuegoEvent, CanceladoEvent, PostulacionAceptadaEvent, PostulacionRechazadaEvent, NuevoScrimDisponibleEvent y ReporteConductaRegistradoEvent.
Requerimientos relacionados	RF2 búsquedas favoritas y alertas; RF7 notificaciones por eventos; escalabilidad con colas; disponibilidad con reintentos; trazabilidad de eventos.

Problema

El sistema genera eventos en muchas partes: creación de scrim, aceptación/rechazo de postulaciones, lobby completo, confirmación total, inicio, finalización, cancelación y reportes de conducta. Cada evento puede interesar a distintos receptores: usuarios, participantes, organizadores, moderadores, sistemas de notificación, alertas de búsqueda favorita, logs de auditoría o futuras colas Kafka/RabbitMQ. Sin Observer, la clase Scrim o ScrimLifecycleService llamaría directamente a EmailNotifier, PushNotifier, DiscordNotifier, BusquedaFavorita y ModerationService. Eso acopla dominio con infraestructura y obliga a modificar lógica de negocio cada vez que se agrega un nuevo receptor.

Solución aplicada en el diseño

- DomainEventBus actúa como sujeto/observable. Tiene una lista de subscribers y métodos subscribe/unsubscribe/publish. En el draw.io figuran suscribe y un suscribe; conviene corregirlo a subscribe y unsubscribe para evitar errores de nomenclatura.
- IObservable define onEvent(event: DomainEvent): void. NotificationObserver y AlertaBusquedaObserver implementan esa interfaz.
- NotificationObserver recibe eventos de dominio y, con ayuda de INotifierFactory, crea o usa los canales de notificación correspondientes.

- `AlertaBusquedaObserver` escucha `ScrimCreadoEvent` o `NuevoScrimDisponibleEvent`, compara el scrim contra las `BusquedaFavorita` guardadas y dispara alertas si coinciden juego, región, rango, rol y horario.
- Los eventos deben ser objetos inmutables con `eventId`, `occurredAt`, `scrimId` y datos mínimos necesarios. Esto facilita la auditoría y evita que los observers consulten demasiadas entidades del dominio.

Flujo de ejemplo dentro de eScrims

12. `ScrimLifecycleService` cambia un estado.
13. Se crea un `DomainEvent`, por ejemplo `LobbyArmadoEvent(scrimId, participantesIds)`.
14. `DomainEventBus.publish(event)` recorre sus observers.
15. `NotificationObserver` genera notificaciones multicanal.
16. `AlertaBusquedaObserver` ignora ese evento si no le corresponde, o procesa solo eventos de nuevo scrim disponible.
17. En una implementación escalable, `publish` puede encolar el evento y los observers procesarlo de forma asincrónica.

Beneficios técnicos

Desacopla el dominio de los canales de comunicación. Permite agregar Slack, SMS, auditoría o analytics sin modificar Scrim. Facilita colas y procesamiento asincrónico. Mejora trazabilidad porque todos los eventos pasan por un flujo explícito. Permite que `BusquedasFavoritas` funcionen como un suscriptor natural del dominio.

Relación con SOLID y GRASP

- SOLID: Observer refuerza SRP porque Scrim o `ScrimLifecycleService` no tienen que ocuparse de enviar emails, push, Discord, alertas de búsquedas favoritas y logs. Su responsabilidad termina al publicar un evento de dominio. OCP se ve al agregar nuevos observers sin modificar la clase que genera el evento.

- SOLID: DIP aparece porque el dominio publica DomainEvent y los subscribers implementan IObserver; no depende de proveedores concretos. ISP se favorece si IObserver solo tiene onEvent(event) y no obliga a implementar métodos de canales que no corresponden. Esto evita que BusquedaFavorita tenga que conocer APIs de email o Discord.
- GRASP: Observer aplica Indirection mediante DomainEventBus, que actúa como intermediario entre eventos del dominio y acciones secundarias. También aplica Low Coupling porque los emisores no conocen receptores concretos. Pure Fabrication justifica DomainEventBus: no es una entidad de negocio del eSport, pero existe para mejorar diseño, trazabilidad y extensibilidad. Protected Variations protege la cantidad y tipo de suscriptores, que seguramente crecerán.

Patrón Abstract Factory

Aspecto	Detalle
Clases UML	INotifierFactory, DevNotifierFactory, ProdNotifierFactory, Notifier, PushNotifier, EmailNotifier, DiscordNotifier y NotificationObserver como consumidor de la factory.
Requerimientos relacionados	RF7 notificaciones multicanal; RNF de disponibilidad ante fallos de proveedores; pruebas sin spam/costos; mención de Abstract Factory en la consigna.

Problema si no se aplica el patrón

El sistema necesita crear familias de objetos relacionados: notificadores push, email y Discord/Slack. Además, esos objetos no deberían ser iguales en desarrollo que en producción. En desarrollo conviene crear notificadores fake o de consola; en producción deben existir adaptadores reales a Firebase, JavaMail/SendGrid y Discord webhook. Si el código usa `new EmailNotifier` o `new DiscordNotifier` en muchas partes, queda atado a clases concretas y a proveedores externos. Sin Abstract Factory, cambiar de entorno o proveedor obliga a modificar puntos de creación dispersos. También se vuelve difícil testear porque las pruebas podrían enviar mails reales o depender de credenciales.

Solución aplicada en el diseño

- `INotifierFactory` define `crearPushNotifier()`, `crearEmailNotifier()` y `crearDiscordNotifier()`, todos retornando `Notifier`.
- `DevNotifierFactory` retorna implementaciones de prueba que registran mensajes en consola, memoria o logs. Esto permite ejecutar tests de integración sin proveedores externos.
- `ProdNotifierFactory` retorna notificadores reales. `EmailNotifier` puede usar `INotificadorEmail` implementado por `JavaMailAdapter` o `SendGridAdapter`; `DiscordNotifier` usa `INotificadorDiscord` implementado por `DiscordAdapter`; `PushNotifier` puede encapsular Firebase.

- NotificationObserver no instancia notifikadores directamente. Solicita la familia correcta a la factory según entorno, región o configuración.
- Si el sistema requiere multirregión, la factory puede incorporar región para elegir proveedores cercanos o plantillas localizadas.

Flujo de ejemplo dentro de eScrims

18. La app arranca leyendo APP_ENV=dev o prod.
19. Se inyecta DevNotifierFactory o ProdNotifierFactory.
20. Cuando llega un evento, NotificationObserver pide a la factory el canal correspondiente.
21. El notifier envía o simula el envío.
22. El resto del dominio no sabe qué proveedor se usó.

Beneficios técnicos

- Centraliza la creación de objetos relacionados.
- Permite cambiar entorno con configuración, no con cambios de código.
- Reduce costos y efectos colaterales en pruebas.
- Trabaja muy bien junto con Observer: el observer decide notificar y la factory decide con qué implementación concreta.
- Aísla credenciales y configuración de proveedores en infraestructura.

Relación con SOLID y GRASP

- SOLID: Abstract Factory aplica DIP porque la aplicación trabaja con INotifierFactory y Notifier, no con Firebase, SendGrid, JavaMail, Discord o Slack directamente. También aplica SRP porque

la creación de notificadores queda en la factory y el envío queda en cada notifier. El dominio no decide qué clase instanciar ni qué credenciales usar.

- SOLID: OCP aparece cuando se agregan familias nuevas de objetos, por ejemplo una ProdLatamNotifierFactory, ProdBrasilNotifierFactory o TestNotifierFactory. LSP obliga a que una eScrim - Fundamentación de Patrones de Diseño DevNotifierFactory pueda reemplazar a una ProdNotifierFactory sin que NotificationObserver cambie su código. ISP recomienda no crear una interfaz gigante si algunos entornos no soportan todos los canales; se puede dividir por canal si crece demasiado.
- GRASP: Creator es el principio más fuerte: la factory es responsable de crear objetos relacionados porque centraliza la información de entorno, región y proveedor. También hay Pure Fabrication, ya que la factory no representa un concepto del negocio, sino una solución técnica para mantener cohesión y bajo acoplamiento. Protected Variations protege contra cambios de proveedores y cambios entre dev/prod.

Patrón Builder

Aspecto	Detalle
Clases UML	ScrimBuilder y Scrim. En el UML, Usuario usa crearScrim(builder: ScrimBuilder): Scrim.
Requerimientos relacionados	RF3 Creación de scrim; validaciones de juego, formato, cupos, roles, región, rangos, latencia, fecha/hora y modalidad.

Problema concreto si no se aplica el patrón

Crear un Scrim válido requiere muchos datos y varias reglas interdependientes. Juego y formato determinan cupos y roles posibles; rangoMin no puede superar rangoMax; región debe ser válida; fechaHora debe ser futura; latenciaMax debe ser positiva; y el organizador debe existir. Un constructor gigante con muchos parámetros sería ilegible y propenso a errores de orden. Por ejemplo, confundir rangoMin con rangoMax o región con juego puede crear objetos inválidos. Usar setters públicos para construir el objeto paso a paso también es riesgoso porque el Scrim puede quedar parcialmente armado y circular por el sistema antes de ser válido.

Solución aplicada en el diseño

- ScrimBuilder ofrece una API fluida: conJuego, conFormato, conRango, conRegion, conLatenciaMax, conFechaHora y build.
- Cada método carga una parte del objeto y retorna el propio builder para encadenar llamadas legibles.
- build() concentra la validación final de invariantes y recién entonces devuelve un Scrim válido.
- Usuario.crearScrim(builder) delega la construcción, pero la responsabilidad de garantizar consistencia queda en ScrimBuilder.
- Conviene que ScrimBuilder inicialice el estado en BuscandoJugadoresState al crear el scrim, porque la consigna indica que todo scrim nuevo inicia en Buscando jugadores.

Flujo de ejemplo dentro de eScrims

23. El organizador selecciona juego, formato, rangos, región, latencia y fecha. eScrims - Fundamentación de Patrones de Diseño
24. La UI o el controller arma un ScrimBuilder.
25. El builder valida invariantes en build.
26. Si todo es válido, devuelve Scrim con estado inicial BuscandoJugadoresState.
27. Si hay error, devuelve una excepción de validación clara.

Beneficios técnicos

- Evita constructores telescópicos.
- Hace la creación más legible y menos propensa a errores.
- Centraliza validaciones de invariantes.
- Permite agregar campos opcionales sin romper constructores existentes.
- Mantiene Scrim sin setters públicos innecesarios.

Relación con SOLID y GRASP

- SOLID: Builder contribuye a SRP porque concentra la construcción y validación inicial de Scrim en ScrimBuilder, evitando constructores enormes o setters públicos que dejen al objeto inconsistente. OCP aparece porque agregar un campo opcional, como contraseña de lobby o reglas especiales, puede hacerse con un nuevo método fluido sin romper llamadas existentes.
- SOLID: ISP se ve en que el builder expone solo pasos de construcción, no operaciones de ciclo de vida, matchmaking o notificación. DIP no es el principio central del patrón, pero puede respetarse si el builder recibe validadores abstractos o servicios de configuración por interfaz cuando las validaciones crecen.

- GRASP: Builder aplica Creator porque tiene todos los datos necesarios para crear un Scrim válido. También funciona como Pure Fabrication: no es una entidad del dominio final, pero mejora la claridad del modelo. Information Expert se cumple parcialmente porque build() centraliza invariantes de creación: formato, cupos, rango, región, fecha futura y duración. High Cohesion se logra porque Scrim queda más limpio y no se llena de lógica de armado paso a paso.

Patrón Command

Aspecto	Detalle
Clases UML	LobbyCommand, InvitarJugadoresCommand, SwapJugadoresCommand, AsignarRolCommand y CommandHistoryInvoker.
Requerimientos relacionados	RF6 Gestión de equipos y roles; endpoint POST /api/scrims/{id}/acciones/{command}; undo antes de confirmar; trazabilidad de acciones de organización.

Problema si no se aplica el patrón

Durante LobbyArmado el organizador puede modificar el lobby: asignar roles, invitar jugadores o hacer swap. Son acciones reversibles mientras el scrim no esté confirmado. eScrims - Fundamentación de Patrones de Diseño Si esas acciones fueran métodos directos dispersos en Scrim o en varios services, no habría historial uniforme, no sería sencillo deshacer y la auditoría quedaría repartida. La consigna pide Command para acciones como AsignarRol, InvitarJugador y SwapJugadores, por lo que el patrón no es decorativo: responde a una necesidad de undo y trazabilidad.

Solución aplicada en el diseño

- LobbyCommand define ejecutar(scrim: Scrim) y deshacer(scrim: Scrim).
- AsignarRolCommand guarda usuario, rolAnterior y rolNuevo. Al ejecutar cambia el rol; al deshacer restaura rolAnterior.
- SwapJugadoresCommand guarda jugador1 y jugador2. Al ejecutar intercambia posiciones/equipo; al deshacer ejecuta el swap inverso.
- InvitarJugadoresCommand agrega un usuario al lobby o lista de espera. Al deshacer lo remueve o lo devuelve al estado anterior.
- CommandHistoryInvoker actúa como invoker/historial. Ejecuta comandos, los guarda en una pila/lista y deshace el último comando en orden inverso.

- Cada comando puede guardar metadatos: organizerId, timestamp, scrimId, motivo y resultado. Eso permite auditoría.

Flujo de ejemplo dentro de eScrims

28. El organizador llama al endpoint de acción.
29. El controller crea el comando concreto a partir del request.
30. ScrimCommandHistory.ejecutar(comando, scrim) ejecuta y guarda el comando.
31. Si el organizador se equivoca, llama a deshacer.
32. El historial toma el último comando y ejecuta deshacer(scrim).

Beneficios técnicos

- Habilita undo real antes de confirmar el lobby.
- Uniforma acciones distintas bajo una misma interfaz.
- Facilita logs de auditoría.
- Permite agregar nuevos comandos sin modificar ScrimCommandHistory.
- Hace que cada acción sea testeable de forma independiente.

Relación con SOLID y GRASP

- SOLID: Command aplica SRP porque cada acción del lobby se encapsula en una clase con una responsabilidad clara: asignar rol, invitar jugador o hacer swap. OCP se cumple porque agregar CambiarCapitanCommand, ExpulsarJugadorCommand o ReemplazarSuplenteCommand no obliga a modificar el Invoker ni Scrim si la interfaz se mantiene estable.
- SOLID: LSP exige que todos los comandos puedan ejecutarse y deshacerse de manera uniforme mediante ScrimCommand. DIP se cumple porque el Invoker depende de la abstracción ScrimCommand y no de comandos concretos. ISP se favorece si la interfaz solo incluye

ejecutar/deshacer; si se agregaran métodos de auditoría, validación o serialización obligatorios, convendría separarlos.

- GRASP: Command aplica Polymorphism porque el Invoker no pregunta qué tipo de acción es; solo invoca ejecutar o deshacer. También aplica Indirection: el controller no modifica directamente el lobby, sino que crea un comando y lo delega al Invoker. Protected Variations protege el conjunto de acciones posibles del lobby, que puede crecer. High Cohesion se mantiene porque cada comando guarda los datos necesarios para revertirse.

Patrón Adapter

Aspecto	Detalle
Clases UML	IAdapterAuth, AdapterSteam, AdapterDiscord, AdapterRiot, OAuthManager, INotificadorEmail, JavaMailAdapter, SendGridAdapter, INotificadorDiscord, DiscordAdapter, JavaMail, SendGrid, Webhook y, conceptualmente, ICalAdapter para calendario.
Requerimientos relacionados	RF1 OAuth opcional; RF7 canales externos; RF10 sincronización iCal; RNF disponibilidad y reintentos ante proveedores externos.

Problema si no se aplica el patrón

eScrims se integra con servicios externos que no hablan el mismo idioma que el dominio. Steam, Riot y Discord tienen SDKs y OAuth distintos; JavaMail y SendGrid tienen APIs diferentes; Discord webhook exige su propio payload; iCal requiere formato RFC 5545. Si el dominio conoce directamente esas APIs, cualquier cambio de SDK o proveedor rompe el código de negocio. Además, las pruebas se vuelven frágiles porque dependen de servicios externos. El problema no es solamente técnico: el dominio necesita conceptos simples como autenticar(usuario) o enviar(mensaje, destino), no detalles de tokens, headers HTTP, payloads JSON o credenciales.

Solución aplicada en el diseño

- IAdapterAuth define autenticar(usuario): Boolean. AdapterSteam, AdapterDiscord y AdapterRiot traducen esa operación simple al flujo real de cada SDK.
- OAuthManager mantiene autenticador: IAdapterAuth y delega la autenticación sin conocer el proveedor concreto.
- INotificadorEmail define enviar(mensaje, destino). JavaMailAdapter traduce a JavaMail; SendGridAdapter traduce a SendGrid.
- INotificadorDiscord define enviar(mensaje, destino). DiscordAdapter traduce al formato de Webhook.
- ICalAdapter, aunque no se ve claramente en el draw.io actual, debería incluirse si se fundamenta RF10: recibe datos del scrim y genera un evento compatible con calendario.
- Los adapters son el lugar natural para timeouts, reintentos, manejo de errores del proveedor y traducción de excepciones externas a errores propios del sistema.

Flujo de ejemplo dentro de eScrims

33. El usuario elige iniciar sesión con Riot.
34. OAuthManager usa AdapterRiot.
35. AdapterRiot ejecuta el flujo requerido por RiotSDK y devuelve Boolean o un usuario autenticado.
36. Para email, EmailNotifier llama a INotificadorEmail.enviar.
37. SendGridAdapter transforma el mensaje a la API de SendGrid sin exponer esos detalles al dominio.

Beneficios técnicos

- Aísla al dominio de SDKs externos.
- Facilita cambiar proveedores sin reescribir reglas de negocio.

- Permite mocks/fakes en pruebas.
- Ubica reintentos y manejo de errores donde corresponde.
- Hace que las interfaces internas sean estables aunque cambien servicios externos.

Relación con SOLID y GRASP

- SOLID: Adapter es una aplicación directa de DIP. El sistema no debe depender de SDKs externos de Steam, Riot, Discord, SendGrid o del estándar iCal, sino de interfaces propias como IAdapterAuth, INotificadorEmail, INotificadorDiscord o ICalendarAdapter. Esto evita que cambios externos contaminen el dominio.
- SOLID: SRP se cumple porque cada adapter se ocupa de traducir una integración específica. SteamAuthAdapter no debería enviar emails; SendGridAdapter no debería validar reglas del scrim; ICalAdapter no debería decidir si un scrim puede iniciar. OCP se cumple porque migrar de SendGrid a Mailgun o agregar GoogleCalendarAdapter implica crear otro adapter sin tocar el dominio.
- GRASP: Adapter aplica Protected Variations porque las APIs externas son puntos altamente variables. También aplica Indirection y Low Coupling: el adapter queda como intermediario entre el modelo propio y el contrato externo. Pure Fabrication justifica adapters como clases técnicas que no existen en el negocio del eSport, pero son necesarias para separar infraestructura de dominio. En particular, ICalAdapter debe verse en el UML si se fundamenta RF10.

Patrón Chain of Responsibility

Aspecto	Detalle
Clases UML	ModerationHandler, AutoModerationHandler, BotModerationHandler, HumanModerationHandler, ModerationService, ReporteConducta, Penalidad, TipoPenalidad, Ban, Kick, Mute y Moderator.
Requerimientos relacionados	RF9 Moderación y penalidades; reportes de conducta; strikes y cooldown; resolución/derivación automática, bot y moderador humano; trazabilidad.

Problema si no se aplica el patrón

Los reportes de conducta no tienen todos la misma gravedad. Un abandono aislado puede resolverse automáticamente; insultos reiterados pueden ser tratados por reglas de bot; acoso grave, smurfing o apelaciones requieren moderador humano. Sin Chain of Responsibility, ModerationService tendría un gran if/else que decide gravedad, tipo de sanción, strikes y escalamiento. Agregar un nuevo nivel, como revisión por comité, obligaría a modificar ese servicio. eScrim - Fundamentación de Patrones de Diseño

La moderación necesita ser trazable y flexible. En torneos oficiales se podría requerir revisión humana para todos los casos; en scrims casuales se podría automatizar más.

Solución aplicada en el diseño

- ModerationHandler tiene next: ModerationHandler, setNext(handler) y handle(report).
- AutoModerationHandler procesa casos simples: primer no-show, primer abandono, cooldown automático o strike básico. Si no corresponde, delega.
- BotModerationHandler procesa patrones intermedios con reglas: mutePlayer, kickPlayer o ban temporal mediante TipoPenalidad.
- HumanModerationHandler crea una tarea para Moderator y adjunta contexto: reporte, historial, strikes, evidencia y scrim relacionado.
- ModerationService solo conoce el primer handler y llama procesar(reporte). La cadena decide internamente quién resuelve.

- Penalidad conserva el resultado aplicado: tipo, motivo, strikes y cooldownHasta.

Flujo de ejemplo dentro de eScrims

38. Un usuario reporta conducta.

39. Se crea ReporteConductaRegistradoEvent y/o se llama a ModerationService.

40. AutoModerationHandler revisa si es caso simple.

41. Si no puede resolver, delega a BotModerationHandler. 42. Si el bot no puede o la gravedad supera umbral, delega a HumanModerationHandler. 43. El moderador aprueba o rechaza el reporte y se registra Penalidad si corresponde.

Patrón Facade

Aspecto	Detalle
Clases UML	
Requerimientos relacionados	RF3 Creacion de Scrims. RF6 Gestion de equipos y roles, comandos. RF8 Estadísticas y feedback. RF9 Moderacion y penalidades, reportes de conducta.

Problema si no se aplica el patrón

El sistema cuenta con varios subsistemas encargados de la creación y gestión de vida del Scrim, gestión del Lobby, envío de reportes por parte del Jugador o Carga de estadísticas por parte del Jugador organizador del Scrim. De no aplicar el patrón Facade, el Jugador sería el responsable directo de ejecutar estas acciones, generando un alto acoplamiento de este con las clases correspondientes a los subsistemas. Era necesario encontrar una forma de delegar todas estas acciones para liberar al Jugador de tantas responsabilidades. El patrón Facade actúa de mediador entre un cliente y múltiples subsistemas, probando ser una sólida solución.

Solución aplicada en el diseño

- ScrimFacade: Gestiona todo lo relacionado al ciclo de vida del Scrim, ejecutando la lógica correspondiente según el caso.
- LobbyFacade: Gestiona todo lo relacionado a equipos y roles dentro del lobby creado, trabaja en conjunto con el patrón Command mencionado anteriormente.
- ModeracionFacade: Gestiona todo lo relacionado a reportes y rating, acciones que realiza el usuario y que son luego gestionadas por el módulo de Moderación.

Flujo de ejemplo dentro de eScrims

- El organizador decide intercambiar de equipos a dos jugadores
- Ejecuta el comando a través de LobbyFacade
- LobbyFacade delega a CommandHistoryInvoker
- CommandHistoryInvoker ejecuta SwapJugadoresCommand y guarda

- El SwapJugadoresCommand delega a GestorLobby
- GestorLobby realiza el swap
- Si el Organizador se arrepiente, ejecuta el deshacerComando a través de LobbyFacade nuevamente, se repite el proceso.

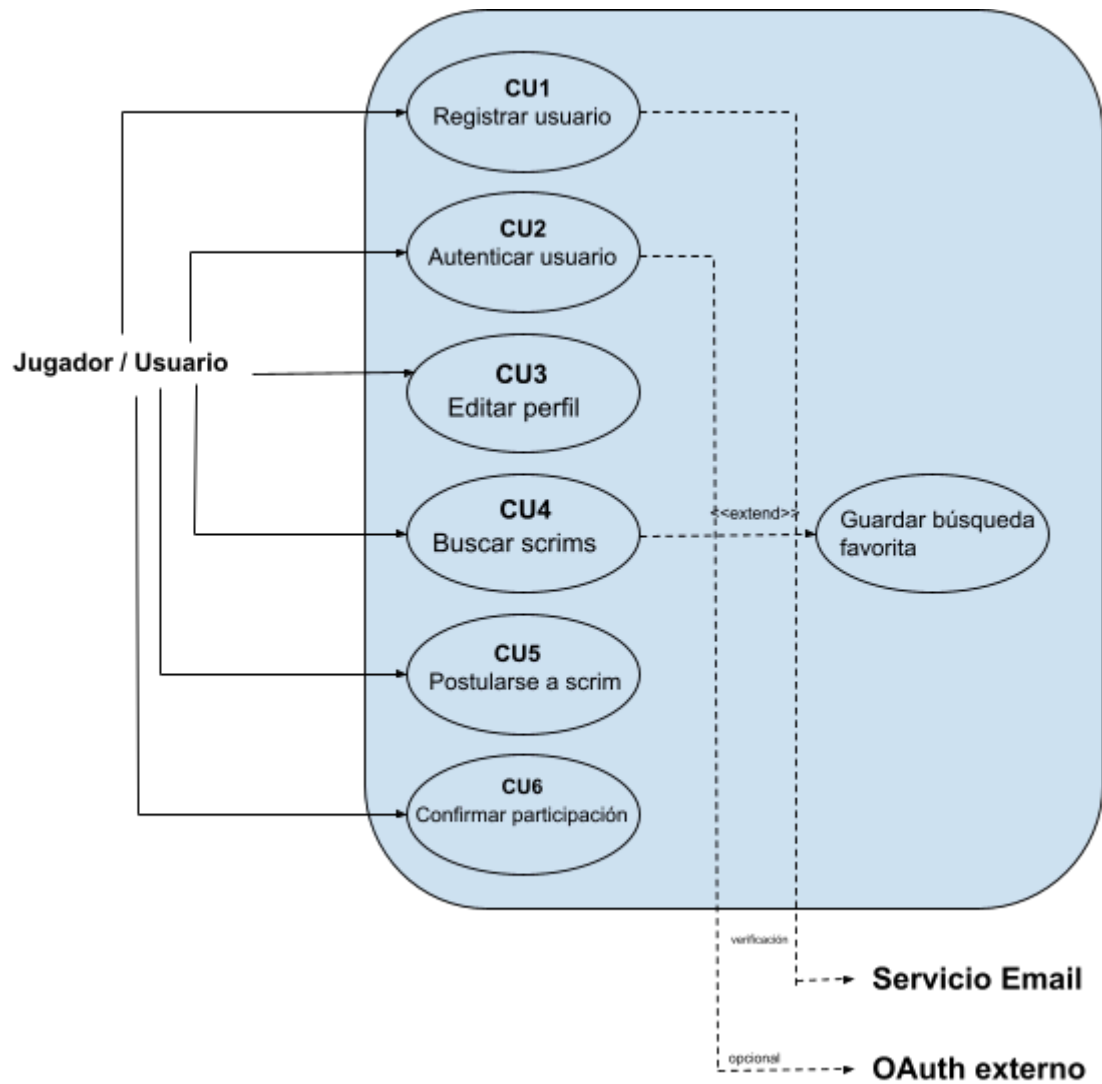
Beneficios técnicos

- Desacopla a Jugador de las múltiples responsabilidades que podría tener.
- Se divide en varios Facades con responsabilidades claras.

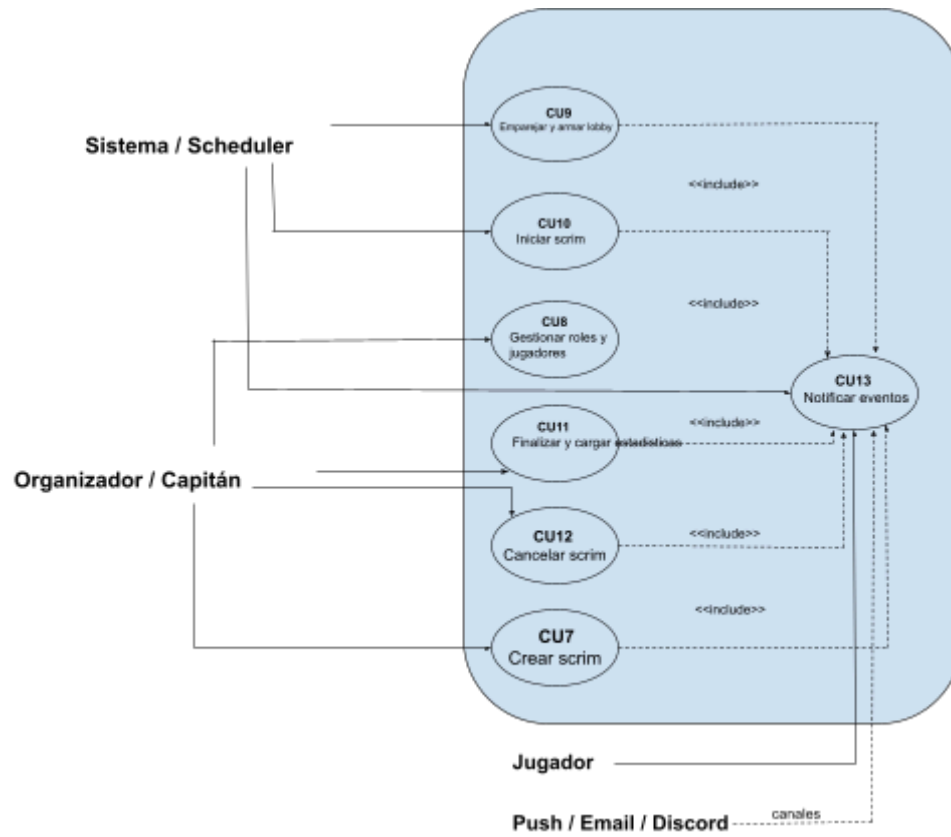
Relación con SOLID y GRASP

Casos de Uso

Sistema eScrims - Gestión de Usuario y Búsqueda de Scrims



Sistema eScrims — Gestión del scrim y notificaciones



Sistema eScrims — Casos de uso de moderación y reportes

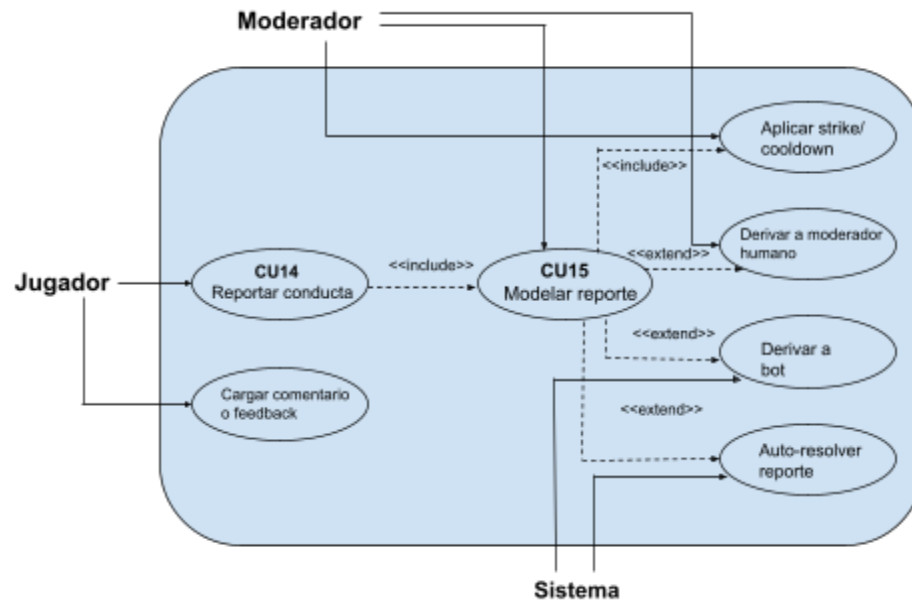


Diagrama general del caso de uso

link al miro: https://miro.com/app/board/uXjVHK5x9Jc=?share_link_id=68866749743

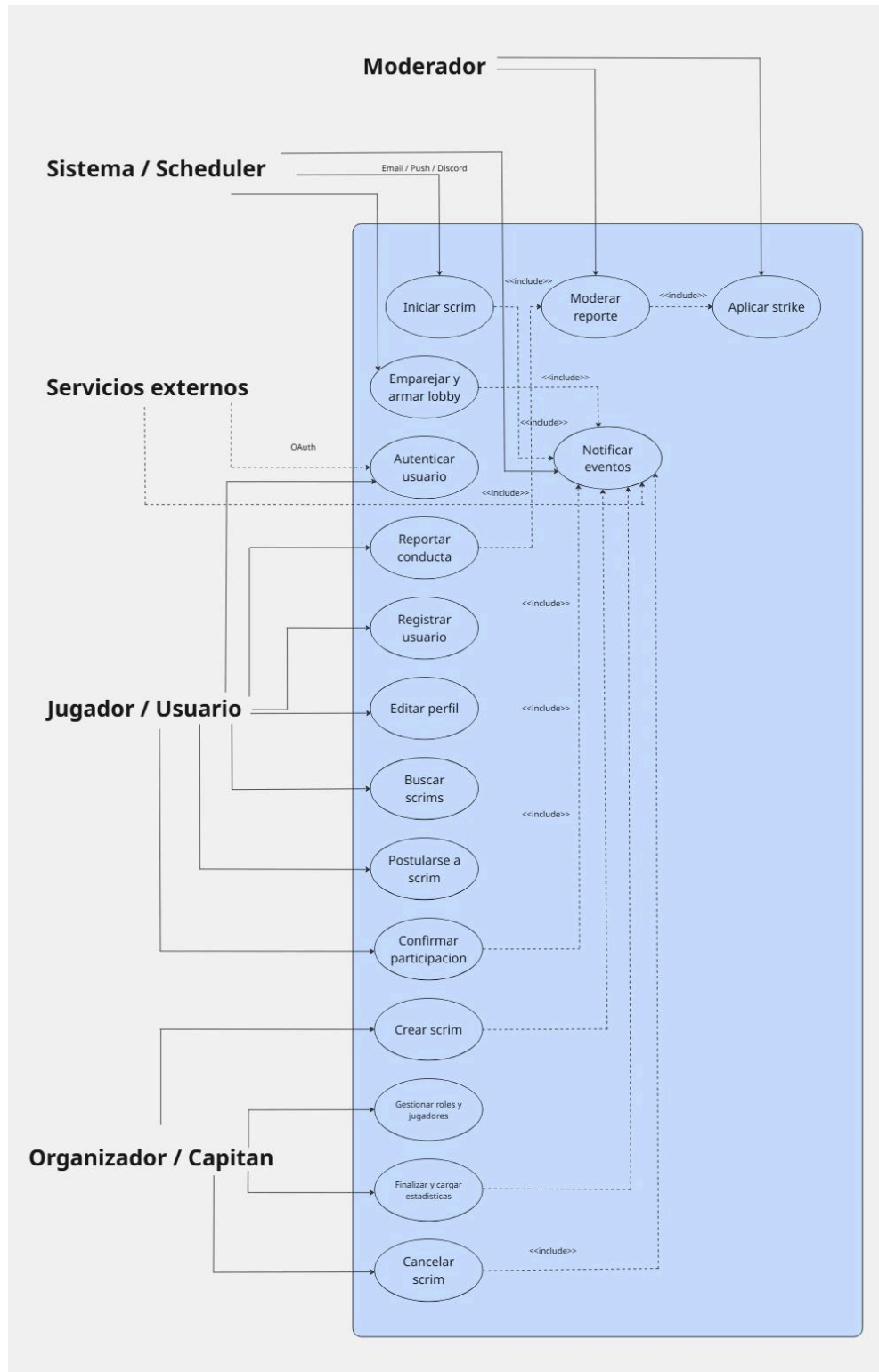
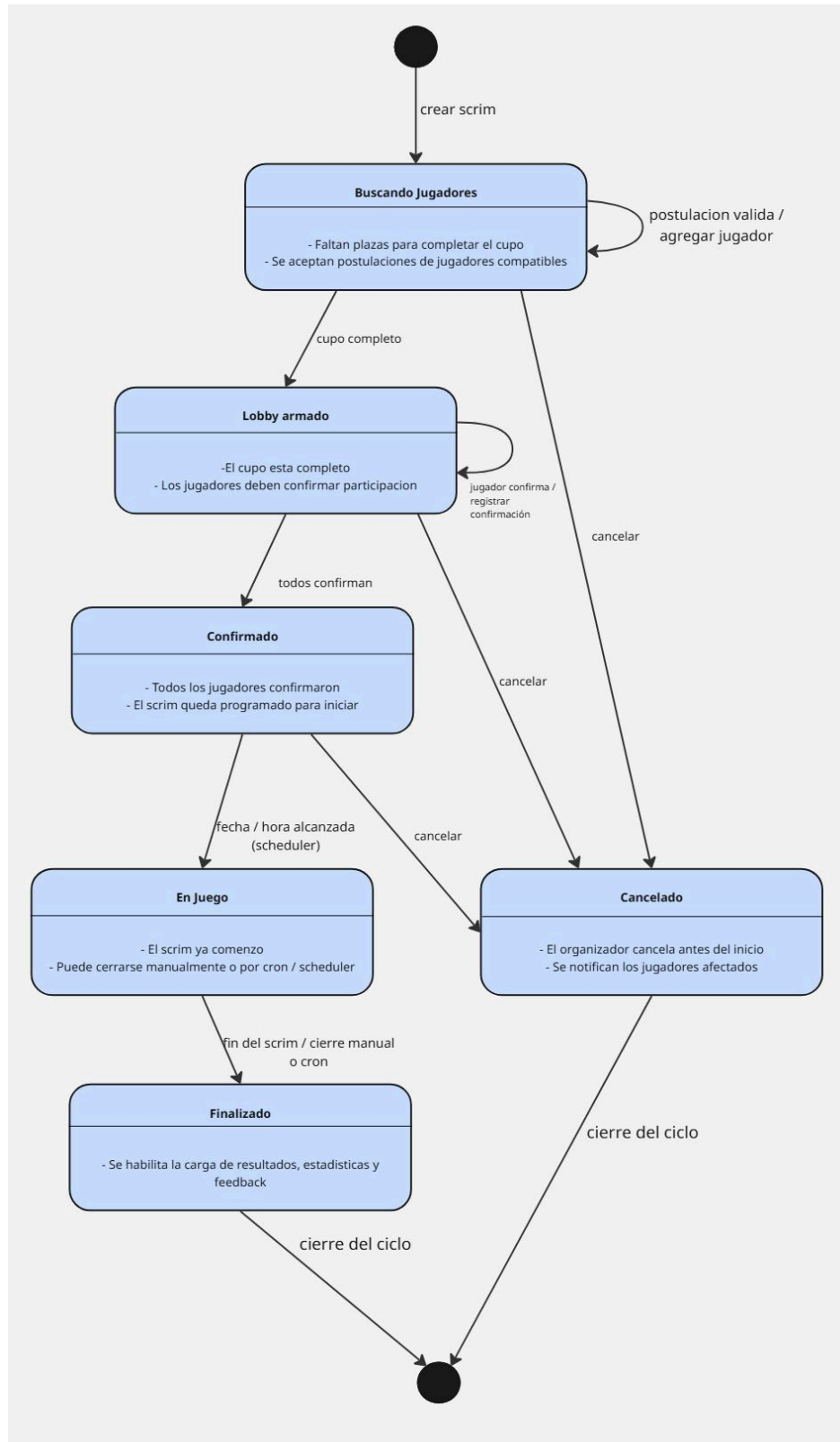


Diagrama de Estados - Ciclo de vida de un Scrim



Historias de Usuario - Plataforma eScrim

HU1 - Registro de usuario

Como: Como jugador

Quiero: Quiero registrarme con usuario, email y contraseña

Para: Para acceder a la plataforma y participar en scrims

Criterios de aceptación:

- Si los datos son válidos, el sistema crea la cuenta
- Si el email ya existe, el sistema lo informa
- La cuenta queda pendiente a verificación

HU2 - Autenticación de usuario

Como: Como usuario registrado

Quiero: Quiero iniciar sesión en la plataforma

Para: Para acceder a mis funciones dentro del sistema

Criterios de aceptación:

- Con credenciales correctas, el sistema permite ingresar
- Con credenciales incorrectas, el sistema muestra error
- Puede autenticarse OAuth externo

HU3 - Edición del perfil

Como: Como jugador

Quiero: Quiero editar mi juego principal, rango, roles, región y disponibilidad

Para: Para que el sistema me recomiende scrims compatibles

Criterios de aceptación:

- El sistema guarda cambios válidos
- Usa esos datos en búsquedas y emparejamiento
- Si hay datos inválidos, solicita corregirlos

HU4 - Búsqueda de scrims

Como: Como jugador

Quiero: Quiero buscar scrims por juego, formato, rango, región, fecha y latencia.

Para: Para encontrar partidas compatibles con mi nivel y conexión

Criterios de aceptación:

- El sistema muestra resultados compatibles
- Si no hay resultados, lo informa
- Permite ver el detalle de scrim

HU6 - Creación de scrim

Como: Como organizador o capitán

Quiero: Quiero crear un scrim indicando juego, formato, cupos, región, rango, latencia, fecha y modalidad

Para: Para organizar una partida amistosa equilibrada

Criterios de aceptación:

- Si faltan datos obligatorios, no se crea
- Si los datos son válidos, el sistema crea el scrim
- El scrim inicia en estado "Buscando Jugadores"

HU5 - Guardar búsqueda favorita

Como: Como jugador

Quiero: Quiero guardar búsquedas favoritas

Para: Para recibir alertas cuando aparezcan scrims compatibles

Criterios de aceptación:

- La búsqueda queda almacenada
- Si aparece una coincidencia, llega una alerta
- La búsqueda puede desactivarse

HU7 - Postulación a un scrim

Como: Como jugador

Quiero: Quiero postularme a un scrim disponible

Para: Para participar de una partida acorde a mi nivel, región y rol

Criterios de aceptación:

- Si cumpla los requisitos, el sistema registra mi postulación
- Si mi rango no coincide, la postulación se rechaza
- Si la latencia supera el máximo, no soy admitido

HU8 - Emparejamiento automático

Como: Como sistema

Quiero: Quiero emparejar jugadores según rango, latencia o historial

Para: Para armar lobbies equilibrados y compatibles

Criterios de aceptación:

- El sistema aplica estrategia configurada
- La estrategia puede ser por MMR, latencia o historial
- Al completar el cupo, el scrim pasa a "Lobby armado"

HU10 - Inicio automático de scrim

Como: Como sistema

Quiero: Quiero iniciar automáticamente el scrim cuando llega la fecha y hora programada

Para: Para respetar el ciclo de vida del entreno

Criterios de aceptación:

- Si el scrim está conformado y llega el horario pasa a "En juego"
- Si fue cancelado no se inicia
- El cambio de estado notifica a los participantes

HU9 - Confirmacion de participacion

Como: Como jugador seleccionado

Quiero: Quiero confirmar mi participación

Para: Para asegurar mi lugar antes de que empiece el scrim

Criterios de aceptación:

- El sistema registra mi confirmación
- Si todos confirman, el scrim pasa a "Confirmado"
- Si alguien no confirmó, puede liberarse el cupo

HU11 - Finalización y carga de estadísticas

Como: Como organizador

Quiero: Quiero finalizar el scrim y cargar estadísticas

Para: Para registrar el resultado, desempeño y feedback

Criterios de aceptación:

- Al finalizar, el scrim pasa a "Finalizado"
- Las estadísticas quedan asociadas al scrim y a los jugadores
- Los comentarios pueden quedar pendientes de moderación

HU12 - Cancelación de scrim

Como: Como organizador

Quiero: Quiero cancelar un scrim antes de que comience

Para: Para evitar que los jugadores esperen una partida que no se realizará

Criterios de aceptación:

- Si el scrim aún no comenzó, puede cancelarse
- Si ya está en juego, no puede cancelarse
- Al cancelarse el sistema notifica a los jugadores afectados

HU14 - Gestión de roles y jugadores

Como: Como organizador o capitán

Quiero: Quiero asignar roles, invitar jugadores o intercambiar jugadores antes del inicio

Para: Para organizar correctamente la composición del equipo

Criterios de aceptación:

- El sistema actualiza el lobby cuando asigno un rol
- Permite realizar swaps antes del inicio
- Si el scrim ya empezó, no permite notificaciones

HU13 - Notificación de eventos

Como: Como jugador

Quiero: Quiero recibir notificaciones por push, email o discord

Para: Para enterarme de eventos importantes relacionados con mis scrims

Criterios de aceptación:

- Si se completa un lobby recibo una notificación
- Si todos confirman los participantes son notificados
- Si el scrim se cancela, todos reciben el aviso

HU15 - Lista de espera y suplentes

Como: Como jugador

Quiero: Quiero quedar en lista de espera cuando un scrim está completo

Para: Para tener oportunidad de ingresar si otro jugador se baja

Criterios de aceptación:

- Si el scrim está completo puedo entrar en una lista de espera
- Si un jugador abandona se notifica al primer suplente
- Si acepto ingreso al lobby

HU16 - Reporte de conducta

Como: Como jugador

Quiero: Quiero reportar conductas indebidas, abandono o no-show

Para: Para que el sistema revise el comportamiento de otros usuarios

Criterios de aceptación:

- El sistema registra el reporte
- El reporte inicia en estado "Pendiente"
- Si falta información puede rechazarse

HU18 - Sincronización con calendario

Como: Como jugador

Quiero: Quiero sincronizar mis scrims con un calendario externo

Para: Para recordar fecha y hora de mis partidas

Criterios de aceptación:

- El sistema genera una entrada compatible con calendario
- Si cambia el horario, el calendario puede actualizarse
- Si el scrim se cancela, el evento puede marcarse como cancelado

HU17 - Moderación de reportes

Como: Como moderador

Quiero: Quiero revisar reportes de conducta

Para: Para aprobarlos, rechazarlos o aplicar sanciones

Criterios de aceptación:

- El moderador puede aprobar, rechazar o derivar el caso
- Si hay reincidencia, pueden aplicarse strikes
- Los casos simples pueden resolverse automáticamente